Московский авиационный институт
(национальный исследовательский университет)
Институт «Компьютерные науки и прикладная математика»

КУРСОВОЙ ПРОЕКТ

по дисциплине «Теория автоматов и формальных языков»

Часть 1

Вариант задания: паттерн №4 из раздела 8 («Проверка пользовательского ввода»)

Паттерн: `if (!Regex.Match(zipCodeTextBox.Text, @"^\d{5}$").Success)`

Регулярное выражение: ^\d{5}$

Описание языка: цепочки из ровно пяти десятичных цифр (ZIP-код)

Москва — 2026

1. Проектирование регулярной грамматики

Заданный паттерн ^\d{5}$ описывает регулярный язык L над алфавитом V_T = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}, порождающий цепочки, состоящие ровно из пяти цифр.

По определению, правая линейная грамматика G = (V_T, V_N, P, S) содержит продукции вида A -> aB или A -> a, где a принадлежит V_T, A, B принадлежат V_N.

Определим формальные компоненты грамматики G:

* V_T = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9} — терминальный алфавит;
* V_N = {S, A, B, C, D} — множество нетерминальных символов;
* S — целевой (начальный) символ грамматики.

Множество правил вывода (продукций) P:

Нетерминал	Правила вывода
S	S -> 0A | 1A | 2A | 3A | 4A | 5A | 6A | 7A | 8A | 9A
A	A -> 0B | 1B | 2B | 3B | 4B | 5B | 6B | 7B | 8B | 9B
B	B -> 0C | 1C | 2C | 3C | 4C | 5C | 6C | 7C | 8C | 9C
C	C -> 0D | 1D | 2D | 3D | 4D | 5D | 6D | 7D | 8D | 9D
D	D -> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Здесь запись вида «S -> 0A | 1A | ... | 9A» является компактной формой записи для десяти отдельных правил: S -> 0A, S -> 1A, ..., S -> 9A.

Семантика нетерминалов (счётчик оставшихся шагов):

* S — требуется распознать ещё 5 цифр;
* A — требуется распознать ещё 4 цифры;
* B — требуется распознать ещё 3 цифры;
* C — требуется распознать ещё 2 цифры;
* D — требуется распознать последнюю 1 цифру (завершающие правила).

Пример цепочки вывода для строки «12345»:

S => 1A => 12B => 123C => 1234D => 12345

Поскольку все правила принадлежат к виду A -> aB или A -> a, грамматика G является правой регулярной (тип 3 по Хомскому).

2. Преобразование грамматики в конечный автомат (КА). Анализ ДКА / НДКА

Построенная правая линейная грамматика преобразуется в конечный автомат $M = (Q, \Sigma, \delta, q_0, F)$ по каноническому алгоритму:

1. Каждому нетерминалу из V_N сопоставляется состояние автомата:

$S \to q_0$, $A \to q_1$, $B \to q_2$, $C \to q_3$, $D \to q_4$

2. Добавляется новое допускающее (финальное) состояние q_f: $F = \{q_f\}$.

3. Для правил вида $X \to aY$ задаётся переход $\delta(X, a) = Y$.

4. Для правил вида $X \to a$ задаётся переход $\delta(X, a) = q_f$.

5. Для обеспечения полной определённости функции переходов вводится состояние-ловушка q_{err}.

Характеристики полученного автомата $M = (Q, \Sigma, \delta, q_0, F)$:

* $Q = \{q_0, q_1, q_2, q_3, q_4, q_f, q_{err}\}$ — множество состояний;
* $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ — входной алфавит (далее обозначен символом d);
* q_0 — начальное состояние;
* $F = \{q_f\}$ — множество допускающих состояний.

Таблица переходов конечного автомата:

Состояние	Символ d (0-9)
$\to q_0$	q_1
q_1	q_2
q_2	q_3
q_3	q_4
q_4	q_f *
* q_f	q_{err}
q_{err}	q_{err}

Проверка на детерминированность:

Автомат называется детерминированным (ДКА), если:

1. В нём отсутствуют ε-переходы;

2. Для любого состояния q принадлежит Q и любого входного символа d принадлежит Σ функция δ(q, d) однозначно определяет не более одного следующего состояния.

В полученном автомате для каждого состояния и каждого символа d существует ровно один переход. Следовательно, автомат уже является ДКА. Выполнение алгоритма детерминизации (НДКА -> ДКА методом подмножеств) не требуется.

Диаграмма переходов ДКА:

-> (q_0) --[d]--> (q_1) --[d]--> (q_2) --[d]--> (q_3) --[d]--> (q_4) --[d]--> ((q_f)) --[d]--> (q_{err}) --[d]--> (q_{err})

Обозначения: -> — вход в начальное состояние; (( )) — допускающее состояние; [d] — переход по любой цифре d принадлежит {0..9}.

Примеры трассировки работы ДКА:

1) Вход «12345» (ровно 5 цифр):

q_0 --[1]--> q_1 --[2]--> q_2 --[3]--> q_3 --[4]--> q_4 --[5]--> q_f [Цепочка допущена: q_f принадлежит F]

2) Вход «1234» (4 цифры — неполная цепочка):

q_0 --[1]--> q_1 --[2]--> q_2 --[3]--> q_3 --[4]--> q_4 [Цепочка отклонена: q_4 не принадлежит F]

3) Вход «123456» (6 цифр — избыточная длина):

q_0 --[1]--> q_1 --[2]--> q_2 --[3]--> q_3 --[4]--> q_4 --[5]--> q_f --[6]--> q_{err} [Цепочка отклонена: q_{err} не принадлежит F]

3. Минимизация ДКА

Процесс минимизации ДКА состоит из двух последовательных этапов: удаление недостижимых состояний и слияние эквивалентных (неразличимых) состояний.

3.1. Алгоритм устранения недостижимых состояний

Состояние q принадлежит Q называется достижимым, если существует хотя бы одно входное слово w принадлежит Σ*, такое что $\delta^*(q_0, w) = q$. В противном случае состояние является недостижимым.

Выполним алгоритм поиска достижимых состояний (BFS) из начального состояния q_0:

* Шаг 0: $U_0 = \{q_0\}$
* Шаг 1: $U_1 = U_0 \cup \delta(q_0, d) = \{q_0, q_1\}$
* Шаг 2: $U_2 = U_1 \cup \delta(q_1, d) = \{q_0, q_1, q_2\}$
* Шаг 3: $U_3 = U_2 \cup \delta(q_2, d) = \{q_0, q_1, q_2, q_3\}$
* Шаг 4: $U_4 = U_3 \cup \delta(q_3, d) = \{q_0, q_1, q_2, q_3, q_4\}$
* Шаг 5: $U_5 = U_4 \cup \delta(q_4, d) = \{q_0, q_1, q_2, q_3, q_4, q_f\}$
* Шаг 6: $U_6 = U_5 \cup \delta(q_f, d) = \{q_0, q_1, q_2, q_3, q_4, q_f, q_{err}\}$
* Шаг 7: $U_7 = U_6 \cup \delta(q_{err}, d) = U_6$ (стабилизация).

Результат: множество достижимых состояний равно всему Q (все 7 состояний достижимы). Недостижимых состояний нет.

3.2. Алгоритм минимизации методом таблицы различимости

Два состояния p и q называются различимыми, если существует цепочка w принадлежит Σ*, которая переводит одно из них в допускающее состояние, а другое — в недопускающее.

Шаг 1 (базовое разбиение). Любая пара состояний (p, q), где одно состояние принадлежит F, а второе не принадлежит F, очевидно различима пустой цепочкой ε (|ε| = 0).

Поскольку $F = \{q_f\}$, а множество нефинальных состояний $Q \setminus F = \{q_0, q_1, q_2, q_3, q_4, q_{err}\}$, все пары (q_f, q_i) для любого $q_i \neq q_f$ помечаются как различимые (символ X).

Шаг 2 (итерационное распространение различимости). Для каждой пары (p, q) проверяем: если для входного символа d пара (δ(p, d), δ(q, d)) уже помечена как различимая, то и пара (p, q) помечается как различимая.

Проверка пар:

* Пара (q_4, q_{err}): $\delta(q_4, d) = q_f$, $\delta(q_{err}, d) = q_{err}$. Пара (q_f, q_{err}) помечена на шаге 1 -> пара (q_4, q_{err}) различима (X).

* Пара (q_3, q_4): $\delta(q_3, d) = q_4$, $\delta(q_4, d) = q_f$. Пара (q_4, q_f) помечена на шаге 1 -> пара (q_3, q_4) различима (X).

* Аналогично по индукции цепочки переходов различают все пары (q_i, q_j) при $i \neq j$, так как состояния q_i и q_j требуют разного количества цифр до попадания в q_f.

Таблица различимости состояний (X — состояния различимы, пустота — диагональ):

	q_0	q_1	q_2	q_3	q_4	q_f
q_1	X					
q_2	X	X				
q_3	X	X	X			
q_4	X	X	X	X		
q_f	X	X	X	X	X	
q_{err}	X	X	X	X	X	X

Вывод по минимизации:

Все пары различных состояний автомата оказались различимыми. Следовательно, ни одна пара состояний не может быть эквивалентна или объединена. Построенный ДКА уже является минимальным.

Минимальный автомат содержит 7 состояний: $q_0, q_1, q_2, q_3, q_4, q_f, q_{err}$.

Этот результат строго согласуется с фундаментальной теоремой теории автоматов: для языка слов фиксированной длины n над алфавитом Σ размерность минимального ДКА составляет в точности n + 2 состояния (n состояний счётчика длины, 1 допускающее состояние и 1 состояние-ловушка для цепочек длины > n).